CSCE 5300 – Introduction to Big Data and Data Science

SQL Concepts

What is Database?

Database is a collection of data.

What is SQL?

SQL is Structured Query Language - used to store, manipulate and retrieve data from RDBMS.

(It is not a database, it is a language used to interact with database)

We use SQL for CRUD Operations :

● CREATE - To create databases, tables, insert tuples in tables etc

● READ - To read data present in the database.

● UPDATE - Modify already inserted data.

● DELETE - Delete database, table or specific data point/tuple/row or multiple rows.

Note - SQL keywords are NOT case sensitive.

Eg: select is the same as SELECT in SQL.

Types of SQL Commands:

1. DQL (Data Query Language) : Used to retrieve data from databases. (SELECT)

2. DDL (Data Definition Language) : Used to create, alter, and delete database objects like tables, indexes, etc. (CREATE, DROP, ALTER, RENAME, TRUNCATE)

3. DML (Data Manipulation Language): Used to modify the database. (INSERT,UPDATE, DELETE)

4. DCL (Data Control Language): Used to grant & revoke permissions. (GRANT,REVOKE)

5. TCL (Transaction Control Language): Used to manage transactions. (COMMIT, ROLLBACK, START TRANSACTIONS, SAVEPOINT)

1. Data Definition Language (DDL)

Data Definition Language (DDL) is a subset of SQL (Structured Query Language) responsible for defining and managing the structure of databases and their objects.

DDL commands enable you to create, modify, and delete database objects like tables, indexes, constraints, and more.

Key DDL Commands are:

● **CREATE TABLE:**

○ Used to create a new table in the database.

○ Specifies the table name, column names, data types, constraints, and more.

Example: CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50), salary DECIMAL(10, 2));

● **ALTER TABLE:**

○ Used to modify the structure of an existing table.

○ You can add, modify, or drop columns, constraints, and more.

Example: ALTER TABLE employees ADD COLUMN email VARCHAR(100);

● **DROP TABLE:**

○ Used to delete an existing table along with its data and structure.

Example: DROP TABLE employees;

● **CREATE INDEX:**

○ Used to create an index on one or more columns in a table.

○ Improves query performance by enabling faster data retrieval.

Example: CREATE INDEX idx_employee_name ON employees (name);

● **DROP INDEX:**

○ Used to remove an existing index from a table.

○ Example: DROP INDEX idx_employee_name;

● **CREATE CONSTRAINT:**

○ Used to define constraints that ensure data integrity.

○ Constraints include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL,and CHECK.

Example: ALTER TABLE orders ADD CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES customers(id);

● **DROP CONSTRAINT:**

○ Used to remove an existing constraint from a table.

○ Example: ALTER TABLE orders DROP CONSTRAINT fk_customer;

● **TRUNCATE TABLE:**

○ Used to delete the data inside a table, but not the table itself.

○ Syntax – TRUNCATE TABLE table_name

2.DATA QUERY/RETRIEVAL LANGUAGE (DQL or DRL)

DQL (Data Query Language) is a subset of SQL focused on retrieving data from databases.

The SELECT statement is the foundation of DQL and allows us to extract specific columns from a table.

● **SELECT:**

The SELECT statement is used to select data from a database.

Syntax: SELECT column1, column2, ... FROM table_name;

Here, column1, column2, ... are the field names of the table.

If you want to select all the fields available in the table, use the following syntax:

SELECT * FROM table_name;

Ex: SELECT CustomerName, City FROM Customers;

● **WHERE:**

The WHERE clause is used to filter records.

Syntax: SELECT column1, column2, ... FROM table_name WHERE condition;

Ex: SELECT * FROM Customers WHERE Country='Mexico';

Operators used in WHERE are:

= : Equal

> : Greater than

< : Less than

>= : Greater than or equal

<= : Less than or equal

<> : Not equal.

Note: In some versions of SQL this operator may be written as !=

● **AND, OR and NOT:**

- The WHERE clause can be combined with AND, OR, and NOT operators.

- The AND and OR operators are used to filter records based on more than one condition:

- The AND operator displays a record if all the conditions separated by AND are TRUE.

- The OR operator displays a record if any of the conditions separated by OR is TRUE.

- The NOT operator displays a record if the condition(s) is NOT TRUE.

Syntax: SELECT column1, column2, ... FROM table_name WHERE condition1 AND condition2 AND condition3 ...;

SELECT column1, column2, ... FROM table_name WHERE condition1 OR condition2 OR

condition3 ...;

SELECT student_id, student_name, age, grade FROM students WHERE age > 18 AND grade >= 80;

● **DISTINCT:**

Removes duplicate rows from query results.

Syntax: SELECT DISTINCT column1, column2 FROM table_name;

ORDER BY:

The ORDER BY clause allows you to sort the result set of a query based on one or more columns.

Basic Syntax:

- The ORDER BY clause is used after the SELECT statement to sort query results.

- Syntax: SELECT column1, column2 FROM table_name ORDER BY column1 [ASC|DESC];

● **GROUP BY**

The GROUP BY clause in SQL is used to group rows from a table based on one or more columns.

Syntax:

- The GROUP BY clause follows the SELECT statement and is used to group rows based on specified columns.

- Syntax: SELECT column1, aggregate_function(column2) FROM table_name GROUP BY column1;

○ Aggregation functions (e.g., COUNT, SUM, AVG, MAX, MIN) are often used with GROUP BY to calculate values for each group.

● **AGGREGATE FUNCTIONS**

These are used to perform calculations on groups of rows or entire result sets. They provide insights into data by summarising and processing information.

Common Aggregate Functions:

- COUNT():

Counts the number of rows in a group or result set.

- SUM():

Calculates the sum of numeric values in a group or result set.

- AVG():

Computes the average of numeric values in a group or result set.

- MAX():

Finds the maximum value in a group or result set.

- MIN():

Retrieves the minimum value in a group or result set.

○ Example: SELECT department, AVG(salary) FROM employees GROUP BY department;

3. DATA MANIPULATION LANGUAGE

Data Manipulation Language (DML) in SQL encompasses commands that manipulate data within a database. DML allows you to insert, update, and delete records, ensuring the accuracy and currency of your data.

● **INSERT:**

- The INSERT statement adds new records to a table.

- Syntax: INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

- Example: INSERT INTO employees (first_name, last_name, salary) VALUES ('John', 'Doe', 50000);

● **UPDATE:**

- The UPDATE statement modifies existing records in a table.

- Syntax: UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;

- Example: UPDATE employees SET salary = 55000 WHERE first_name = 'John';

● **DELETE:**

- The DELETE statement removes records from a table.

- Syntax: DELETE FROM table_name WHERE condition;

- Example: DELETE FROM employees WHERE last_name = 'Doe';

JOINS

In a DBMS, a join is an operation that combines rows from two or more tables based on a related column between them.

Joins are used to retrieve data from multiple tables by linking them together using a common key or column.

Types of Joins:

1. Inner Join

2. Outer Join

3. Cross Join

4. Self Join

1) Inner Join

An inner join combines data from two or more tables based on a specified condition, known as the join condition.

The result of an inner join includes only the rows where the join condition is met in all participating tables.

It essentially filters out non-matching rows and returns only the rows that have matching values in both tables.

Syntax:

SELECT columns FROM table1 INNER JOIN table2 ON table1.column = table2.column;

2) Outer Join

Outer joins combine data from two or more tables based on a specified condition, just like inner joins. However, unlike inner joins, outer joins also include rows that do not have matching values in both tables.

Outer joins are particularly useful when you want to include data from one table even if there is no corresponding match in the other table.

Types:

There are three types of outer joins: left outer join, right outer join, and full outer join.

1. Left Outer Join (Left Join):

A left outer join returns all the rows from the left table and the matching rows from the right table.

If there is no match in the right table, the result will still include the left table's row with NULL values in the right table's columns.

Example:

SELECT Customers.CustomerName, Orders.Product

FROM Customers

LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;

2. Right Outer Join (Right Join):

A right outer join is similar to a left outer join, but it returns all rows from the right table and the matching rows from the left table.

If there is no match in the left table, the result will still include the right table's row with NULL values in the left table's columns.

Example: Using the same Customers and Orders tables.

SELECT Customers.CustomerName, Orders.Product

FROM Customers

RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;

3. Full Outer Join (Full Join):

A full outer join returns all rows from both the left and right tables, including matches and non-matches.

If there's no match, NULL values appear in columns from the table where there's no corresponding value.

Example: Using the same Customers and Orders tables.

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```